POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Embedded Systems project research report

OpenGL: desktop version comparison and analysis with respect to Safety Critical

Federico Costantini, Jacopo Donnini

Advisors:
Prof. William Fornaciari
Prof. Federico Reghenzani

Academic year:
2024-2025

Abstract: The goal of the project is to find what are the main differences between the default Desktop version of OpenGL, and its Safety Critical version, both from a coding point of view and especially performance-wise. OpenGL (Open Graphics Library) is a cross-platform, language-agnostic API for rendering 2D and 3D vector graphics. It defines a standardized set of functions that applications can call to exploit the GPU's computation capabilities, and allowing to render images or scenes in both 2D and 3D. However, certain contexts and application uses require more safety, predictability, and certification: the Safety Critical version comes into play, having a restricted subset of calls and functions that guarantees a more stable and deterministic behavior of the application. In this report, we present the differences between the two versions, both from a theoretical perspective and recreating test cases to pinpoint the core aspects of the implementations.

Key-words: OpenGL, Desktop, Embedded Systems, Safety Critical

1. Introduction

OpenGL is the industry standard API for real-time 2D and 3D graphics, offering a rich feature set, from immediate mode drawing and a fixed-function pipeline in its early days, to fully programmable shaders and a wide extension ecosystem today. It is a specification developed and maintained by the Khronos Group. Its Desktop profile exposes hundreds of functions, dynamic state changes, and optional extensions that allow developers to squeeze every last drop of performance out of modern GPUs.
However, in safety-critical domains (avionics, medical imaging, automotive dashboards, etc.), unpredictability is unacceptable. OpenGL SC ("Safety-Critical") was born due to the way the dynamism of the full Desktop API can introduce risks: unbounded state changes at runtime, unpredictable shader linking order, and optional extensions whose behavior can vary between drivers. By carving out a well-defined subset, eliminating deprecated immediate-mode calls, display lists, feedback mechanisms, and other sources of nondeterminism, SC ensures that every draw call and shader link behave exactly as written, facilitating static analysis and certification.
Our goal in comparing Desktop versus SC is to quantify exactly what has been removed or constrained and to measure the performance and code complexity trade-offs. The next sections will walk through the API differences we uncovered, the tests we conducted in both environments, and highlight where Desktop flexibility gives way to SC's safety guarantees.

1.1. General OpenGL Information

The main aspects of OpenGL are:

- **Specification versus implementation**: the OpenGL specification defines exactly how each function must behave and what output it must produce. Hardware and driver vendors (e.g. GPU manufacturers) provide concrete implementations that conform to the spec, allowing different drivers to vary internally as long as the observable results match. From the point of view of the programmers, the OpenGL specification is a black-box standard. This obviously leads to a faster and easier implementation of a working environment, with a slight loss of flexibility. It is up to the card manufacturer to ensure that certain quality standards are met.
- **Core profile vs. immediate mode**:
  - *Immediate mode* (legacy): High-level fixed-function pipeline; simple to use, but inefficient and deprecated.
  - *Core profile*: Programmable pipeline with modern shader support, buffer objects, and explicit state management for performance and flexibility.
- **Extensions**: Vendors expose new features (advanced rendering techniques, optimizations) via optional extensions in drivers. Applications query and enable supported extensions at run-time.
- **State machine model**: OpenGL operates as a large state machine (the *context*). State-changing commands (e.g. `glPolygonMode`, buffer bindings, ...) configure how subsequent draw calls behave. Thus functions are divided into state-changing and state-using, the latter being functions that perform some operations based on the current state of the program.
- **Object abstractions**: OpenGL uses opaque 'objects' (e.g. buffers, textures, shaders) to group and manage subsets of state. Each object encapsulates its own parameters, much like a C struct, but managed by the driver.

The following code snippet will help clarify what has been listed above:

```
int main(){
    // initialization section
    glGenBuffers(1, &VBO);
    glBindBuffer(GL_ARRAY_BUFFER, VBO); // STATE-CHANGE
    glBufferData(GL_ARRAY_BUFFER, sizeof(data), data, GL_STATIC_DRAW);  // STATE-USE
    [...]
    // draw section
    glBindBuffer(GL_ARRAY_BUFFER, VBO)
    glDraw(GL_TRAINGLES, 6, GL_UNSIGNED_INT, 0);
}
```

The code snippet shows a simple creation of a Vertex Buffer Object, along with its use in the rendering section of the program. Whilst minimal, the example helps us identify some core aspects of OpenGL:

- **glBindBuffer** is a state-changing function. It tells the GPU that any operation concerning an Array Buffer, from that point of the code until explicitly set otherwise, will be performed with respect to the bound VBO. **glBufferData** is instead a state-using function that copies the data passed as parameter into the VBO that is bound to the GPU at the moment of execution (more exactly, it can also behave as a state-changing function, but it is not its primary usage).
  It is important to notice that in the call for binding data from the CPU to the GPU, we specified **GL_STATIC_DRAW**. This command helps the GPU in placing the data in a certain memory region that is more compliant with the number of access that is to be expected with respect to the data itself (in this case, static means that data will not be changed throughout the program). While these aspects are encouraged to be taken into consideration, they do not deprive programmers of the ability to treat the data in other ways to what was stated before. However, this usually results in a poorly optimized program
- **glDraw** function helps us better understand the "black-box" approach: we define the primitive we want the GPU to render (the primitive is the main geometrical shape that the GPU outputs from the vertices received as input, usually these primitives are either points, lines, or triangles) and the number of primitives to render, but how the GPU performs this operation can't be controlled, except for what is allowed to be modified through shaders (explained in more detail later). We have the guarantee that the output of this operation will be presented in a specific format, depending on the current state of the program.

1.2. OpenGL SC Overview (from Khronos specifications)

OpenGL SC is a safety-critical subset of the OpenGL ES 2.0 API, designed for deterministic, certifiable graphics on embedded devices. The main aspects are:

- **Safety-critical subset of OpenGL ES 2.0**: Builds on the lightweight ES 2.0 API, stripping out non-

deterministic and deprecated features (e.g. object deletion, display lists, feedback) to guarantee predictable behavior.

- **Framebuffer-centric, immediate-mode interface**: All drawing commands render directly into a RAM-backed framebuffer (color, depth, stencil), ensuring every draw call's exact effect is known at compile- and run-time.
- **Deterministic numeric and error semantics**: Requires floating-point accuracy to approximately 1×10^{-5}, bounds buffer-access errors without crashing, and uses a strict GL-error model (errors are ignored on failure and retrievable via `GetError`) to support certification.
- **EGL-friendly context management**: Designed to work with EGL 1.4 (including robustness and surfaceless extensions), though vendor-specific "native" APIs may also be used.

A typical program that uses OpenGL SC begins with calls to open a window into the framebuffer into which the program will draw. Then, calls are made to allocate an OpenGL SC context and associate it with the window. Once a context is allocated, the programmer is free to issue OpenGL SC commands. As already mentioned, a standard procedure to obtain this is to use EGL, an interface between Khronos rendering APIs such as OpenGL ES and the underlying native platform window system.
Let's look at an initialization snippet:

```
    // Initialize window with SDL
    window = SDL_CreateWindow("CUBES ES", 0, 0, WIN_WIDTH, WIN_HEIGHT,
    fullscreen ? (SDL_WINDOW_OPENGL | SDL_WINDOW_FULLSCREEN) : SDL_WINDOW_OPENGL);

    // Initialize EGL
    eglDisplay = eglGetDisplay(EGL_DEFAULT_DISPLAY);
    eglInitialize(eglDisplay, nullptr, nullptr);

    // Configure EGL attributes
    const EGLint configAttributes[] = {
        EGL_SURFACE_TYPE, EGL_WINDOW_BIT,
        EGL_RENDERABLE_TYPE, EGL_OPENGL_ES2_BIT, // Request GLES 2.0
        EGL_RED_SIZE, 8, EGL_GREEN_SIZE, 8,
        EGL_BLUE_SIZE, 8, EGL_ALPHA_SIZE, 8,
        [...],
        EGL_NONE
    };

    eglChooseConfig(eglDisplay, configAttributes, &eglConfig, 1, numConfig);

    // Get native window handle for EGL
    SDL_SysWMinfo sysInfo;
    SDL_VERSION(&sysInfo.version); // Set SDL version
    SDL_GetWindowWMInfo(window, &sysInfo);
    EGLNativeWindowType nativeWindow =          (EGLNativeWindowType)sysInfo.info.x11.
    window;

    // Create EGL Surface and Context
    eglSurface = eglCreateWindowSurface(eglDisplay, eglConfig, nativeWindow, NULL);
    eglContext = eglCreateContext(eglDisplay, eglConfig, EGL_NO_CONTEXT,
    contextAttributes);

    // Make the Context Current
    eglMakeCurrent(eglDisplay, eglSurface, eglSurface, eglContext);
    [...]
    }
```

In the snippet above, we used SDL in parallel with EGL for window creation and input management (not included in the code but used in our tests). SDL is only used to create a cross-platform window and obtain the native handle. EGL then takes over for deterministic context creation and surface management. As stated before, we first need to initialize and configure EGL by "getting" a display and setting all the configuration attributes, like the number of bits used for each color and the version of OpenGL required. Then, we create the context and make it the main one, so that later draw functions will perform their operations onto this framebuffer.
The sequence for initialization is:

1. Get and initialize an EGL display.
2. Choose a configuration with the desired framebuffer format.
3. Create a rendering surface and context.

4. Make the context current, so all subsequent OpenGL SC commands target it.

The main aspect that makes EGL fit OpenGL-SC perfectly is that it handles context creation, surface binding, and integration with native display/framebuffer without the need of a full OS window manager. It is minimal, certified, and made to fit directly into embedded/safety-critical systems. Instead, for Desktop applications, we tend to use a library called GLFW that handles inputs directly (without the need of another library). However, GLFW depends on the window systems of the OS in which it is called and their event loops: things that safety-critical embedded systems typically do not have.

2. Graphic Engine

Here, we briefly explain how we implemented the simple graphics engines we used for our testing.
A Graphics engine is the software layer responsible for translating application data (model, textures, animations, UI elements) into rendered frames displayed on screen. At its core, it manages the interaction between the CPU and GPU, managing resources, executing rendering pipelines, and synchronizing frame updates with the display.
A general overview of a graphic engine is the following:

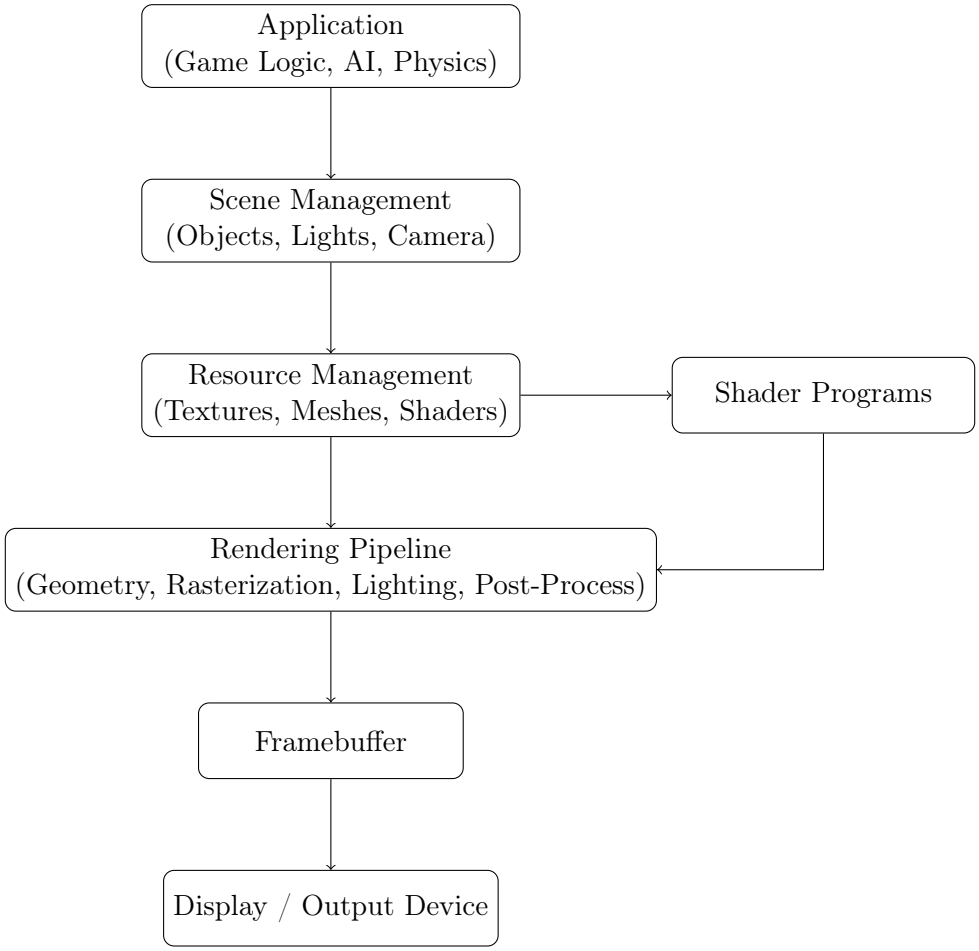

Figure 1: General architecture of a graphics engine.

In our implementation, we followed this scheme closely. Only the first block, Application, is minimal in our engine since we did not need any logical aspect to prepare our scenarios.
There are two main important aspects that are worth mentioning to better grasp the following pieces of code: CPU-GPU data flow and the rendering pipeline.
The rendering process in both OpenGL Desktop and OpenGL-SC follows the same high-level principle: CPU prepares data and issues draw commands, while the GPU executes them to produce the final image. The differences between the two are highlighted in later paragraphs, but in general it is a question of flexibility versus determinism: OpenGL Desktop allows for more dynamic state changes and on-the-fly shader compilation, SC enforces stricter, static data and pipeline definitions to guarantee predictable execution times.
The graphics pipeline is a fundamental concept in computer graphics that describes the process of converting a 3D scene into a 2D image. In essence, it is a series of steps that takes vertex data (the points that make up

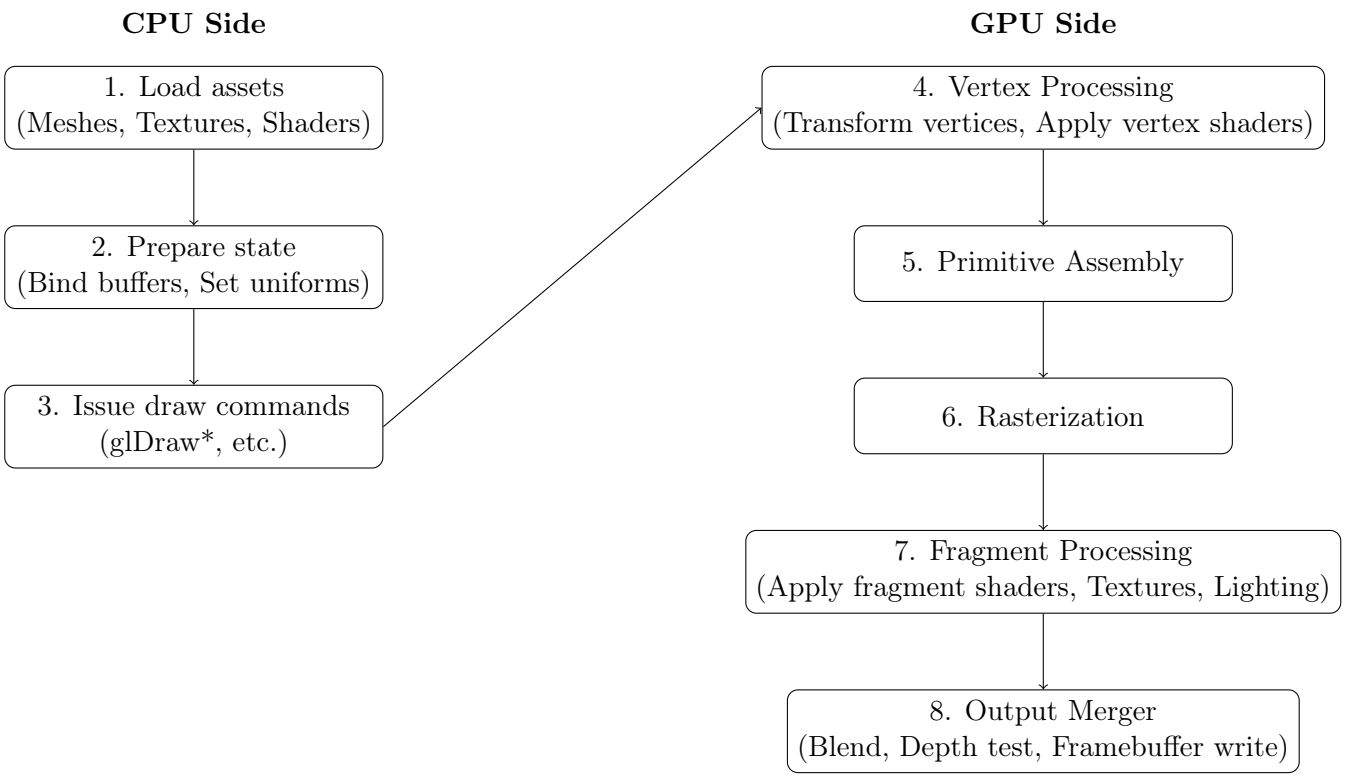

Figure 2: Simplified CPU–GPU data flow in OpenGL and OpenGL SC.

3D models) and transforms it into the final pixels on the screen. In OpenGL, some of these steps are optional, others can only be initiated from a range of predefined options, and others are fully programmable.

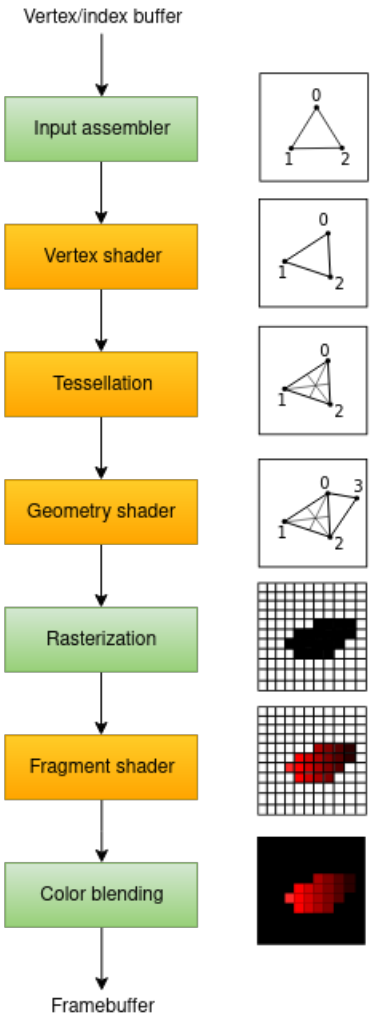

Figure 3: Graphics Pipeline

The two stages that are relevant for us (and fully programmable) are the vertex shader and the fragment shader. The **Vertex Shader** is a programmable stage where you can manipulate the vertices individually. Common operations in the vertex shader include transforming the vertices from their local object space into the 3D world space and then into the 2D screen space.
The **Fragment Shader** is another programmable stage that processes each fragment generated by the rasterizer (a fragment can be thought as a "potential pixel" that contains information like color, depth, and texture coordinates). Here, complex calculations to determine the final color of each pixels are carried out. This is where effects like texturing, lighting, and shadowing are typically implemented.
In our case studies, apart from modifying the vertex and fragment shaders, we did not tamper with the rest of the pipeline (we used compute shaders, but those are explained later on). We implemented the rest of the pipeline in a very basic manner.

3. Key Differences

As mentioned before, OpenGL-SC is, in simple terms, a stripped-down version of OpenGL-ES, made to guarantee safety-critical specifications. Let's first compare the key differences between OpenGL-ES and its SC counterpart, and then focus more in detail on the comparison between OpenGL "standard" and SC.

3.1. OpenGL ES 1.x vs. OpenGL SC 1.0.1

OpenGL ES 1.0 is derived from Desktop OpenGL 1.5 with fixed-point support added; OpenGL SC 1.0.1 is a delta-spec to OpenGL 1.3 tailored for safety certification, removing dynamic features while reintroducing display lists for legacy support.

Feature	OpenGL ES 1.0	OpenGL SC 1.0.1
Memory Management	Allows dynamic allocation	Removed for predictability
Pipeline Type	Fixed-function with limited programmability	Fixed-function only
Error Handling	Standard error reporting	Reduced for deterministic behavior
Display Lists	Removed	Reintroduced for legacy support
Texture Formats	Broad support	Simplified to avoid edge cases
Certification Support	None	Designed for safety certification

Table 1: Comparison: OpenGL ES 1.0 vs. OpenGL SC 1.0.1

3.2. OpenGL ES 2.0 vs. OpenGL SC 2.0

OpenGL ES 2.0 introduces a fully programmable pipeline with GLSL; OpenGL SC 2.0 locks down dynamic behavior, integrates robustness and immutable storage, and targets safety-critical certification.

Feature	OpenGL ES 2.0	OpenGL SC 2.0
Memory Allocation	Dynamic allocation/deletion at runtime	Preallocated at init; no runtime deletion
Pipeline Type	Fully programmable, GLSL shaders	Fully programmable, but with stricter control
Dynamic Shader Compilation	Allowed	Removed for predictability
Error Handling	Standard error reporting	Reduced for deterministic behavior
Texture Management	Flexible formats	Immutable storage via `GL_EXT_texture_storage`
Robustness Support	Optional	Built-in via `GL_KHR_robustness`
Certification Support	None	Designed for safety certification

Table 2: Comparison: OpenGL ES 2.0 vs. OpenGL SC 2.0

3.3. Vertex Array Object

The first main difference between OpenGL and OpenGL-SC concerns VAO. A Vertex Array Object is an OpenGL Object that stores all of the state needed to supply vertex data. It stores the format of the vertex as well as the Buffer Objects providing the vertex data arrays. A VAO merely reference the buffers, it does not copy or freeze their contents; if referenced buffers are modified later, those changes will be seen when using the VAO. VAO allows to quickly switch between vertex formats without re-specifying all the attribute calls.
Usually, a VAO stores pointers to Vertex Buffer Objects (buffers that store the main vertices attributes) and Element Buffer Objects (buffers that indicate which vertices are connected in the specified primitive).
Although VAO are a main aspect of OpenGL, they conflict with SC principles and are therefore not included. VAOs hide attribute and buffer configuration inside an object. When a VAO is bound, a whole set of settings silently changes. This makes it harder to trace exactly what vertex state is active at a given draw call without tracking VAO history. Moreover, VAO can be modified after creation, and this mutability means different runs could use different layouts unless carefully controlled (bad for reproducibility). Without VAOs, the sequence of **glVertexAttribPointer** calls is directly visible in code and trace logs: this helps safety-critical audits and certification processes.
Despite their ease of use, VAO does not directly affect performance in most scenarios. It is only when defining different layouts for the vertices attributes that they come in handy. Otherwise, if data is layered in the same way throughout the program, there is no actual performance gain but only a less bloated code.

The code below shows the main difference between using VAOs and not using them:

```
// OpenGL "Standard"
int main(){
    [...]
    unsigned int cVAO, cVBO, cEBO;
    glGenVertexArrays(1, &cVAO);
    glGenBuffers(1, &cVBO);
    glGenBuffers(1, &cEBO);

    glBindVertexArray(cVAO);

    // Configure vertex buffers + attributes only once
    glBindBuffer(GL_ARRAY_BUFFER, cVBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(data), data, GL_STATIC_DRAW); // copies vertices on the GPU (buffer memory)
    glEnableVertexAttribArray(0);

    // ... Other attributes and element buffer binding (cEBO)

    [...]

    // Render
    glBindVertexArray(cVAO);
    glDrawElements(GL_TRIANGLES, 36, GL_UNSIGNED_INT, 0);
}
```

In the above, we defined **cVAO** as the main Vertex Array Object. The subsequent declared buffers will be accessible from this VAO. In the render loop, we will only need to bind the needed VAO before the draw call in order to correctly bind all the buffers that we want. In simple terms, a VAO is just a wrapper containing multiple buffers with a specific layout.

```
// OpenGL "Safety Critical"
int main(){
    [...]
    glGenBuffers(1, &vbo);
    glBindBuffer(GL_ARRAY_BUFFER, vbo);
    glBufferData(GL_ARRAY_BUFFER, sizeof(data), data, GL_STATIC_DRAW);

    // Configure attributes directly (here pos attribute)
    glVertexAttribPointer(posLoc, 3, GL_FLOAT, GL_FALSE, stride, (void *)0);
    glEnableVertexAttribArray(posLoc);

    // ... Other attributes and index buffer (color, texture, indices)

    [...]

    // Render (must rebind buffers/attributes explicitly)
    glBindBuffer(GL_ARRAY_BUFFER, vbo); // vertex buffer
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, ibo); // index buffer
    glDrawElements(GL_TRIANGLES, 36, GL_UNSIGNED_BYTE, 0);
}
```

The difference between the two codes is easy to understand. As stated before, performance gains from VAOs depend on the frequency of state changes; in scenarios with frequent vertex layout switches, VAOs can significantly reduce CPU overhead. If multiple objects were to be defined that had varying vertices layout specification, we would need to call **glVertexAttribPointer** inside the render loop to redefine the correct layout before drawing the object. Despite not recommended, this approach is made compliant with OpenGL SC in order to make programs run on platforms that have memory constraints: indeed, to avoid this approach, most of the time we would need to pad the data, requiring more memory usage from our system.

3.4. Instance Rendering

Instance Rendering allows to submit one single draw call to render multiple objects with some varying attributes across instances. Instance data (e.g. transforms) is uploaded once to a buffer, and the GPU repeats the draw internally for each instance with `glDrawElementsInstanced`. This reduces CPU overhead by batching many

objects into a single draw (reduces CPU-GPU sync overhead). We later tested this specific aspect in one of our scenarios. Instance rendering is a powerful technique, especially for scenarios in which we have many objects slightly varying between each other (maybe color, or position in space). However, as for VAO, OpenGL-SC does not implement this functionality. Instance rendering has several issues with respect to safety-critical systems.

- Instance rendering is silently performing a loop inside the GPU. The loop count and its internal state changes are not visible in the command stream at the per-instance level. For SC certification, auditors need to see exactly what happens each frame
- Instance redering execution time depends on too many aspects (vertex complexity, attribute changing, driver internals) and is not easily analyzable. Instance rendering complicates WCET analysis in SC systems due to hidden GPU-side loops, requiring specialized tools or vendor data for precise timing.

```
// OpenGL "Standard" -> extracted from first test scenario
[...]
glBindBuffer(GL_ARRAY_BUFFER, instanceVBO);
glBufferSubData(GL_ARRAY_BUFFER, 0, cubes_tot * sizeof(glm::mat4), trans.data());
// the varying data between instances

// One draw call only
glDrawElementsInstanced(GL_TRIANGLES, 36, GL_UNSIGNED_INT, 0, cubes_tot);



// OpenGL SC (no instancing)
[...]
for (int i = 0; i < cubes; i++) {
        // ... Apply transform[i] for this cube
        glDrawElements(GL_TRIANGLES, 36, GL_UNSIGNED_BYTE, 0);  // draw the cube
    }
```

3.5. Graphic Reset Recovery

Certain events can result in a reset of the GL context. After such an event, it is referred to as a lost context, and it is unusable for almost all purposes. Recovery is handled differently between OpenGL standard and SC. In the normal context, all context state is lost and you may have to recreate buffers, textures, shaders, etc. The app can gracefully degrade or restart rather than crash.
OpenGL SC operates in safety-critical systems where an unexpected screen freeze or reset could have disastrous consequences. In the event of a severe, unrecoverable hardware or software fault, the system is designed to enter a defined failure state. The system's safety protocols would then dictate the appropriate action, which might involve:

- Transitioning to a safe, simplified display mode
- Displaying a static warning to the user
- Performing a controlled system restart

In many safety-critical systems, there are dedicated hardware and software mechanisms to monitor the health of the GPU and the graphics driver. These can detect potential issues before they lead to a catastrophic failure, allowing the system to take preemptive action.

3.6. Out-of-bounds Robustness Behavior

In OpenGL SC, commands operating on buffer will detect attempts to read from or write to a location in abound buffer object at an offset less than zero, or greater than or equal to the buffer size. When such an attempt is detected, a GL error is generated. Any command unable to generate a GL error, such as buffer object accesses from the active program, will not read or modify memory outside of the data store of the buffer object and will not result in GL interruption or termination. This behavior prevents memory corruption, undefined actions, or access to other contexts' memory (important for safety certification).
In the Desktop counterpart, out-of-bounds may be undefined behavior: could crash, corrupt memory, or even appear to "work" sometimes. Robustness in Desktop GL is only guaranteed if GL_KHR_robustness extension is explicitly enabled.

3.7. Shader Compilation

A **shader** is a programmable operation which is applied to data as it moves through the rendering pipeline. A key difference between the two version of OpenGL under analysis regards shader compilation. In a "standard" setting, shaders are often compiled at run-time, when actually needed. To give an example, consider a videogame in which you have to travel between different worlds. Each one of these worlds will need a specific set of shaders that will be compiled only when the player is actually about to enter the world. However, this is not possible in an OpenGL-SC context. In this scenario shader compilation is offline-only because run-time shader compilation introduces non-determinism, unbounded execution time, and certification problems. This approach allows SC's contexts to be more predictable during their execution. Moreover, it allows for a faster start-up and usually faster transitioning between shaders' linking. However, it is easy to understand how this requires more memory and can be a burden for systems with low capacity. Despite this, compiled shaders are usually very lightweight and do not stress the memory as other elements do (for example, the .obj files representing modeled objects). Offline compiled shaders guarantee bit-for-bit reproducibility (same binary -> same output on certified hardware).

3.8. Shading Language

Another notable difference between the two versions of OpenGL is the shading language they support. OpenGL-SC relies on a restricted and outdated version of the OpenGL Shading Language (GLSL), which is based on older Desktop OpenGL profiles and lacks many of the modern language features and built-in functions. This is a deliberate design choice: by fixing the language version and avoiding frequent feature updates, the specification reduces the complexity of driver implementations and ensures long-term stability for safety-critical certification. For example, in modern Desktop OpenGL (GLSL 4.6), one can write a fragment shader using explicit layout qualifiers, subroutines, and advanced texture functions:

```
#version 460 core
layout(location = 0) in vec3 normal;
layout(location = 0) out vec4 fragColor;

void main() {
    fragColor = vec4(normalize(normal) * 0.5 + 0.5, 1.0);
}
```

In contrast, OpenGL-SC typically supports only much older versions of GLSL (e.g., GLSL ES 1.00 or 1.20), without advanced qualifiers, subroutines, or newer data types. A similar fragment shader in SC-compatible GLSL would need to use older syntax and implicit variable bindings:

```
#version 100
attribute vec3 color; // instead of layout, we use attribute
varying vec3 normal; // varying refers to a variable obtained from a previous shader or
    destined to a future shader

void main() {
    gl_FragColor = vec4(normalize(normal) * 0.5 + 0.5, 1.0);
}
```

This limitation means that many modern shader programming conveniences available in Desktop OpenGL are not usable in SC, requiring developers to adopt older coding styles and avoid features introduced in recent GLSL revisions. While this restricts flexibility, it ensures that the language remains stable and predictable across long product lifecycles, which is essential for safety-critical environments.

3.9. Additional Differences

Here we list a couple of other smaller dissimilarities:

- **Textures**: no compressed formats unless fixed in specification; no mipmap generation (a mipmap is a precalculated, optimized sequence of images, each of which has an image resolution which is a factor of two smaller than the previous; they are intended to increase rendering speed and reduce aliasing artifacts); only certain texture formats are allowed.
- **Queries**: No occlusion queries, no timer queries (anything that could vary unpredictably is removed).
- **State Management**: no GL_CLAMP wrap mode; only CLAMP_TO_EDGE and REPEAT allowed; no aliases.
- **No implicit allocations**: all resources (textures, buffers) must be allocated at app init, before rendering starts

4. Experiments

To better understand the differences between Desktop OpenGL and its safety-critical subset (OpenGL SC), a series of experiments was conducted. Each experiment explores a different feature or performance aspect of the two APIs, and was conducted isolated from others by creating and managing a separate branch in our code using GitHub (links are available at the end of the report). In all experiments, we tracked the evolution of 5 aspects of the simulation:

- **Memory:** tracking both the total VRAM usage (MB) and the amount of uploads per each simulation frame (kB);
- **FPS:** the frames per second of the running simulation, at each frame;
- **Timings:** an analysis of the frame's timing in different aspects (total/average frame time, CPU render time, GPU wait time);
- **Calls and Triangles:** the amount of *draw()* function calls and triangles simulated per each frame.
- A notable metrics that is often taken into consideration for Safety Critical systems is the **Worst Case Execution Time (WCET)**. Strictly speaking, WCET in real-time systems is theoretical worst-case. In our analysis we included the so called "observed" WCET as the maximum frame time obtained duringtest. Despite this not being the real WCET, it can still be of interest to the reader and can lead to some consideration.

4.0.1 Generalities

In order to conduct our experiments, we first created a simple graphic engine using OpenGL Desktop. To manage windowing system and input control, we exploited GLFW, an Open Source, multi-platform library that provides a simple API for creating windows, contexts and surfaces, receiving inputs and events.
Because OpenGL is only really a standard/specification, it is up to the driver manufacturer to implement the specification to a driver that the specific graphics card supports. Since there are many different versions of OpenGL drivers, the location of most of its function is not known at compile-time and needs to be queried at run-time. AN **OpenGL Loading Library** is a library that loads pointers to OpenGL functions at runtime, core as well as extensions. For this purpose, we GLAD, an open source library.
To carry out matrix and vector operations, we made use of **GLM**. It is a C++ mathematics library for graphics software based on the OpenGL Shading Language (GLSL) specification. GLM provides classes and functions designed and implemented with the same naming conventions and functionalities than GLSL.
In order to load models and relative node structures and texture association, we exploited the **Open Asset Import Library (Assimp)**, a library to load various 3D file formats into a shared, in-memory immediate format.
The aforementioned specifications regard the setup for a standard OpenGL Desktop application. To perfectly run OpenGL-SC, specific hardware is required: the driver for the GPU of the system must expose the OpenGL SC API. These drivers are not included with standard Desktop GPU drivers. They are part of special safety-certified driver packages that hardware vendors usually only ship for automotive, avionics, or industrial embedded platforms.
Given the above, we opted to simulate OpenGL-SC 2.0 behavior by using OpenGL-ES 2.0 API, avoiding using any function or technique not compliant with OpenGL-SC specifications. Since OpenGL-SC is a stripped down version of OpenGL-ES, this approach gave us the possibility to mimic SC behavior as closely as possible. Nonetheless, we are aware of this approximation and treated the data collected accordingly.

4.0.2 Tracker class

In order to retrieve meaningful data in our experiments, we created a class to track programs' execution in the four main aspects cited before. We tailored this class for each specific experiment. Below, is an example of one of the tracker used:

```
class PerfTracker {
public:
    // Timing
    double frameTime, cpuRenderTime, gpuWaitTime, fps;

    // Counters
    int frameCount, drawCalls, trisThisFrame;
    int shaderBinds, textureBinds;

    // Memory
    long long totalVramAllocated, dataUploadedThisFrame;
```

```
12
13      void beginFrame();        // mark frame start, reset counters
14      void beginCpuRender();    // mark CPU render section
15      void endCpuRender();      // measure CPU render time
16      void endFrame();          // finalize frame stats, update FPS
17      void countDrawCall();     // increment counters
18      void trackVramAllocation(long long bytes);
19      void printStats();        // output or log results
20  };
```

This design allowed us to quantify not just frame time but also GPU load, state changes, and memory usage. The full implementation also handles smoothing of frame times via a sliding history window, and can optionally write all statistics into a CSV file for later analysis. The CSV output was later parsed by a Python script to generate graphs and visual comparisons across experiments.
In practice, the workflow of the `PerfTracker` during one frame is as follows:

1. **Initialization:** At program start, the tracker is initialized. If CSV logging is enabled, the output file is opened and a header row is written.
2. **Frame begin (`beginFrame()`):** At the start of each frame, a high-resolution timestamp is taken. Per-frame counters (draw calls, triangles, shader binds, texture binds, VRAM uploads) are reset to zero.
3. **CPU render section (`beginCpuRender()` / `endCpuRender()`):** When the CPU enters the rendering code, another timestamp is recorded. At the end of the CPU render phase, the elapsed time is measured to obtain the CPU-side rendering time.
4. **Frame end (`endFrame()`):** At the end of the frame, a final timestamp is recorded. From this, the total frame time is computed. GPU wait time is estimated as `frameTime - cpuRenderTime`. The tracker also updates running statistics such as average, minimum, maximum frame time, and smoothed FPS over a sliding window.
5. **Counters during rendering:** Throughout the frame, dedicated methods increment counters whenever specific events occur:
   - `countDrawCall()`, the number of draw calls submitted
   - `countTriangles(int)`, the number of triangles rendered
   - `countShaderBind()` / `countTextureBind()`, the state changes tracked
   - `trackVramAllocation()` / `trackDataUpload()`, memory usage monitored
6. **Logging and output:** After finishing a frame, statistics are optionally written into the CSV file. At runtime, `printStats()` can also display current metrics in the console for quick inspection.

This structured cycle gave us precise data per frame, enabling later comparison of CPU vs GPU times, the cost of state changes, and memory overhead in each experiment.

4.0.3 Testing Environments

To carry out our tests, we were restricted to the usage our personal computers.
For the first and third case study, the environment was the following:

- Hardware Model: Lenovo LOQ 15IRX9
- Memory: 16 GiB DDR5
- Processor: Intel Core i7-13650HX
- Graphics: NVIDIA GeForce RTX 4060 Laptop GPU
- Disk: 1.0 TB
- OS: Pop!_OS 22.04 LTS

For the second case study, the environment was the following:

- Hardware Model: Razer blade 15" 2019
- Memory: 16 GiB DDR4
- Processor: Intel Core i7-10750H
- Graphics: NVIDIA® GeForce RTX™ 2060 Ti with Max-Q
- Disk: 512 GB
- OS: Ubuntu 22.04 LTS

4.1. World of Cubes

The first experiment we conducted aimed at exposing:

- Performance differences between instance rendering (OpenGL Desktop) and "normal" rendering (OpenGL-SC).
- Outline the differences in window's management and input handling.

- Stability of the program when under stress.

To achieve these goals, we created a simple scene in which we rendered a large number of cubes spread around the center of the scene, each one rotating differently around their local model center. Each cube is colored on each side with the mixture of two textures. No lighting is taken into consideration in this scenario (see Figure 4).

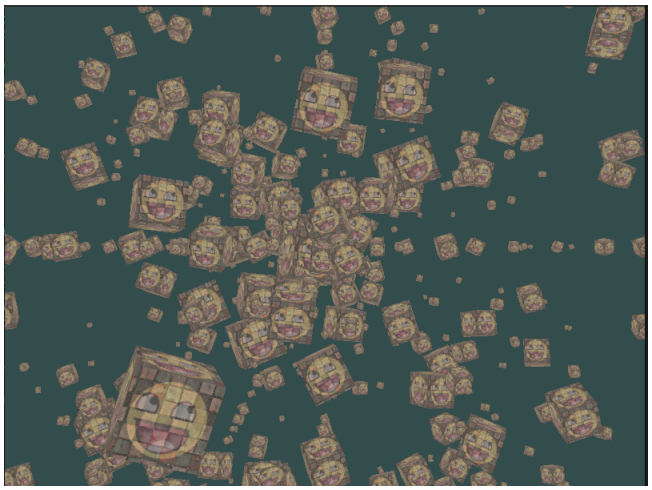

Figure 4: World of Cubes Scene

Regarding windowing management, we previously described how an EGL context is created. For OpenGL Desktop, we used GLFW library and created a context as follows:

```
    // GLFW initialization
    glfwInit();
    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3); // here requested version 3.3
    glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE); // load open gl
    functions with no back compatibility

    window = glfwCreateWindow(WIN_WIDTH, WIN_HEIGHT, "OpenGL", NULL, NULL);
    glfwMakeContextCurrent(window);

    // GLAD initialization: it contains alla the fucntions not available in system
    OpenGL Library
    if (!gladLoadGLLoader((GLADloadproc)glfwGetProcAddress))      return -1;
```

The above code is self-explanatory. Concerning the input management system, we used GLFW for OpenGL Desktop directly and SDL for OpenGL-SC.
The real difference between the two codes lies in the drawing mechanism. As said before, we exploited instance rendering for OpenGL Desktop. For this specific scenario, data is prepared in the following way:

```
    // Verex setup
    glBindVertexArray(cVAO);
    glBindBuffer(GL_ARRAY_BUFFER, cVBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(cube), cube, GL_STATIC_DRAW); // copies
    vertices on the GPU (buffer memory)
    tracker.trackVramAllocation(sizeof(cube));

    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, cEBO);
    glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(cube_indices), cube_indices,
    GL_STATIC_DRAW);
    tracker.trackVramAllocation(sizeof(cube_indices));

    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 8 * sizeof(float), (void*)0);
    glEnableVertexAttribArray(0);
    // color attribute
    glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 8 * sizeof(float), (void*)(3*
    sizeof(float)));
    glEnableVertexAttribArray(1);
    // texture attribute
```

```
    glVertexAttribPointer(2, 2, GL_FLOAT, GL_FALSE, 8 * sizeof(float), (void*)(6*
    sizeof(float)));
    glEnableVertexAttribArray(2);

    glGenBuffers(1, &instanceVBO);
    glBindBuffer(GL_ARRAY_BUFFER, instanceVBO);
    glBufferData(GL_ARRAY_BUFFER, CUBES * sizeof(glm::mat4), nullptr, GL_DYNAMIC_DRAW);

    std::size_t vec4Size = sizeof(glm::vec4);
    for (int i = 0; i < 4; i++) {
        glEnableVertexAttribArray(3 + i);
        glVertexAttribPointer(3 + i, 4, GL_FLOAT, GL_FALSE, sizeof(glm::mat4), (void*)(
    i * vec4Size));
        glVertexAttribDivisor(3 + i, 1);
    }

    glGenBuffers(1, &rotAxisVBO);
    glBindBuffer(GL_ARRAY_BUFFER, rotAxisVBO);
    glBufferData(GL_ARRAY_BUFFER, CUBES * sizeof(glm::vec3), rot.data(), GL_STATIC_DRAW
    );
    glEnableVertexAttribArray(7);
    glVertexAttribPointer(7, 3, GL_FLOAT, GL_FALSE, sizeof(glm::vec3), (void*)0);
    glVertexAttribDivisor(7, 1);

    glBindVertexArray(0);
```

We created two additional Vertex Buffer Objects with respect to "normal" implementation: the instanceVBO that holds info about each single instance of the cubes, and rotAxisVBO which contains the axis around which the cubes rotate. Code of the actual drawing is included in the code snippets inserted before in the Key Difference section.
For the OpenGL-SC implementation, we followed all the limitations previously discussed, and most of the code reflects what can be found in the previous section (thus not included here - refer to the Github repositories for the complete code structure).
In order to put our scenario under stress, we conducted experiments rendering a total of 1k cubes, then 10k, 100k and lastly 1 million. We collected data for a total of ten thousand frames.
Most notably, we can immediately see the difference of the two versions when it comes to Frame per Seconds **FPS**. The OpenGL-SC version is indeed slower than its Desktop counterpart. However, the FPS are more stable in the SC version. This perfectly aligns with our expectations: instance rendering allows our Desktop application to perform a single draw call to the GPU, but it is less predictable and directly manageable with respect to its SC counterpart that uses single draw calls. Below we included the graphs extracted from the test case with 10k cubes. SC implementation averaged 150 FPS less than its Desktop counterpart, which instead shows clear signs of instability in the frame rate. Similar results were achieved with the other tests conducted

Figure 5: FPS vs Frame Number - OpenGL-SC

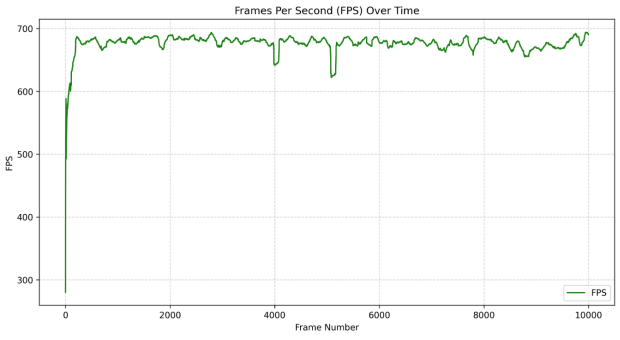

Figure 6: FPS vs Frame Number - OpenGL Desktop

The same conclusions can be extrapolated from the analysis of Frame Timing (Cpu Render time and GPU wait time). Since in OpenGL Desktop we are directly processing all 10k cubes at once, CPU render time is obviously greater. Most noticeably though, it is the instability of the render time in the two versions. It is evident how the Desktop version is less stable. These results confirm that instance rendering, although powerful (in OpenGL Desktop we averaged 7 fps for 1 million cubes, while 5 for OpenGL-SC), doesn't allow for exact timing analysis

for Safety-critical system: analyzing the WCET would be hard when so many operations are masked behind a single procedure without programmers direct control.

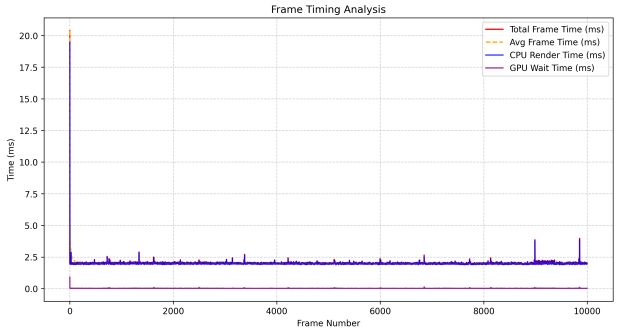

Figure 7: Frame Timing Analysis - OpenGL-SC

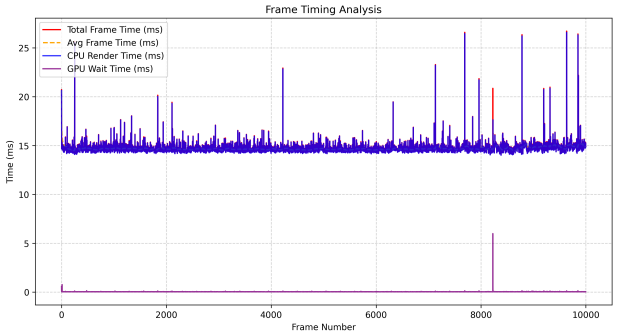

Figure 8: Frame Timing Analysis - OpenGL Desktop

Finally, the **WCET** in the two different versions of the program reflects the theoretical analysis we provided. As stated before, the WCET measure taken is not really reflective of the program as in the desktop version non-determinism and dynamic calls do not allow to be precise, but it still is a good indicator. The results show that while desktop OpenGL achieves far higher average performance, its observed WCET scales poorly when increasing workload (e.g., $\sim 5.8\times$ jump from 10,000 to 100,000 cubes, from 4.59 ms to 26.61 ms). By contrast, OpenGL SC exhibits higher baseline frame times but a much smaller WCET growth ($\sim 1.9\times$, from 20.38 ms to 39.09 ms). This behavior reflects the design choice in OpenGL SC: features like instanced drawing are prohibited because they rely on driver-level and GPU-level optimizations that make runtime execution unpredictable, preventing reliable WCET guarantees.

4.2. Compute Shaders

The second experiment aimed to compare:

- Native compute shader execution (OpenGL Desktop, OpenGL 4.3+).
- Emulated compute execution using fragment shaders (OpenGL-SC 2.0 / OpenGL ES 2.0 subset).
- Differences in data flow, code structure, and limitations.

We chose to render an animated **Menger sponge** fractal, adapted from a *Shadertoy* shader[1]. The effect is a continuous camera fly-through inside the fractal, with fully procedural shading.

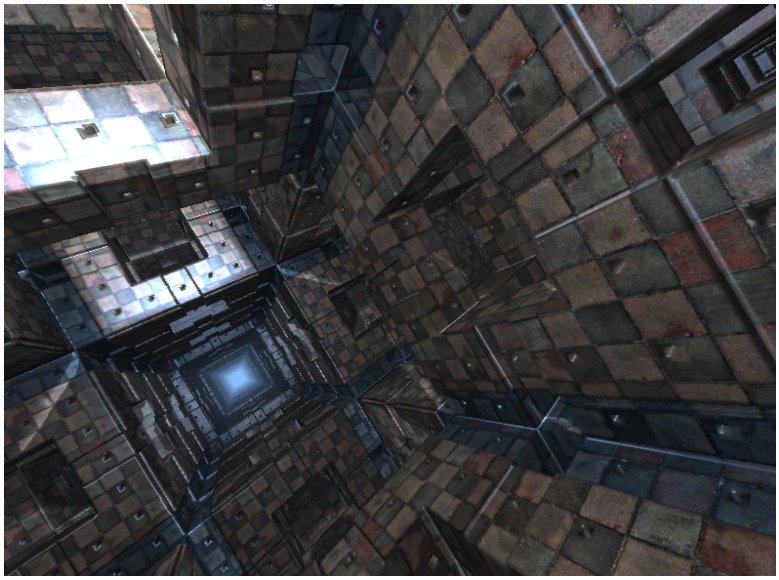

Figure 9: Menger sponge fractal rendered in the compute shader experiment

OpenGL Desktop Implementation

[1] `[https://www.shadertoy.com/view/ldyGWm]`

We implemented the shader as a true `GL_COMPUTE_SHADER`:

1. Create an `RGBA32F` texture bound as image unit 0:

```
glBindImageTexture(0, compTex, 0, GL\_FALSE, 0, GL\_WRITE\_ONLY, GL\_RGBA32F);
```

2. Compile and link the compute shader.
3. Set uniforms (`iTime`, `iResolution`, sampler `iChannel0`).
4. Dispatch work groups and set a memory barrier:

```
glDispatchCompute((width+15)/16, (height+15)/16, 1);
glMemoryBarrier(GL\_SHADER\_IMAGE\_ACCESS\_BARRIER\_BIT);
```

5. Draw a fullscreen quad to display the output texture.

OpenGL-SC / OpenGL ES 2.0 Implementation

Lacking compute shaders, we emulated the algorithm in a fragment shader:

- Requested an ES 2.0 context via EGL + SDL.
- Created an `RGBA8` texture + FBO as the compute output.
- Vertex shader: pass-through position and UV.
- Fragment shader: perform fractal raymarch per-pixel and write color.
- Compute pass: bind FBO, draw fullscreen quad.
- Present pass: draw quad again sampling the result texture.

Here are snippets of how we achieved the same scene in the different OpenGL implementations. The Desktop path uses a GLSL 4.30 `.comp` compute shader, while the SC/ES path uses a GLSL ES 1.00 `.frag` + `.vert` pair to emulate the same logic:

```
#version 430
layout(local_size_x = 16, local_size_y = 16) in;
layout(rgba32f, binding = 0) uniform image2D imgOutput;
...
void main() {
    ivec2 pix = ivec2(gl_GlobalInvocationID.xy);
    if (pix.x >= int(iResolution.x) || pix.y >= int(iResolution.y)) return;
    vec4 color;
    mainImage(color, (vec2(pix) + 0.5) / iResolution * iResolution);
    imageStore(imgOutput, pix, color);
}
```

Listing 1: Excerpt from Desktop .comp shader

Here, `gl_GlobalInvocationID` gives direct pixel coordinates. The shader writes results via `imageStore` into a bound RGBA32F image, enabling arbitrary read/write access without rasterization.

```
// GLSL ES 1.00
precision highp float;
varying vec2 vFragCoord;
...
void main() {
    vec2 fragCoord = vFragCoord * uResolution;
    vec4 outColor;
    mainImage(outColor, fragCoord);
    gl_FragColor = outColor;
}
```

Listing 2: Excerpt from SC/ES .frag shader

Here, no compute stage exists. The `mainImage` logic runs per-fragment in a fullscreen quad render pass, and output is written to `gl_FragColor` in the currently bound FBO.

```
attribute vec2 aPos;
varying vec2 vFragCoord;
void main() {
    vFragCoord = aPos * 0.5 + 0.5;  // [-1,1] becomes [0,1]
    gl_Position = vec4(aPos, 0.0, 1.0);
```

```
}
```

Listing 3: Pass-through vertex shader for SC/ES

This minimal vertex shader exists only to pass normalized coordinates to the fragment stage, emulating the per-pixel addressing that `gl_GlobalInvocationID` provides natively in compute shaders.
The change from compute to fragment execution impacts:

- **Invocation Model:** Compute threads can exit early for out-of-bounds pixels; fragment shaders always run for covered fragments.
- **Output Path:** Compute writes to images directly; fragment shaders write to the framebuffer's color attachment.
- **GLSL Features:** Compute shaders require #version 430+, image load/store, and no precision qualifiers; SC/ES requires #version 100, explicit precision, and uses `texture2D`.

Results

To evaluate the impact of using native compute shaders (Desktop OpenGL) versus a "fake" compute shader implementation (OpenGL SC/ES), we measured frame timings, FPS stability, memory usage, and draw call counts over a run of 10,000 frames. The main findings are summarized below.

Aspect	**Desktop (Native Compute Shader)**	**SC/ES ("Fake" Compute Shader)**
Frame Timing and GPU Wait Time	Lower and more stable total frame time; fewer GPU wait spikes due to direct compute dispatch (`glDispatchCompute`).	Slightly higher and more constant total frame time; GPU wait time consistently larger because each frame processes a full-screen fragment pass.
FPS Stability	Sustained ∼85–90 FPS with low variation; first-frame spike due to shader warm-up.	Sustained ∼75–85 FPS with slightly higher variation; additional rasterization overhead reduces peak FPS.
Memory Usage	∼8.1 MB VRAM allocated due to persistent SSBO usage.	∼2.6 MB VRAM allocated; only requires a single RGBA texture, no SSBO.
Draw Calls and Triangles	1 draw call (final display pass only).	2 draw calls per frame: one for the fake compute pass, one for the final display pass.
WCET (estimate)	15.8884 ms maximum frame time; not a reliable WCET due to runtime optimizations, dynamic calls, and indeterminism.	16.6263 ms maximum frame time; a meaningful WCET estimate since execution is fully defined at compile time and free of dynamic behavior.

Table 3: Summary of observed differences between Desktop OpenGL compute shaders and SC/ES fake compute shaders.

From a theoretical perspective, the key distinction lies in how work is dispatched: native compute shaders execute as general-purpose GPU programs without rasterization overhead, while the SC/ES approach embeds computation in the fragment stage of a full-screen quad render. This difference manifests in higher GPU wait times, lower FPS, and slightly increased draw call counts in the SC/ES version.

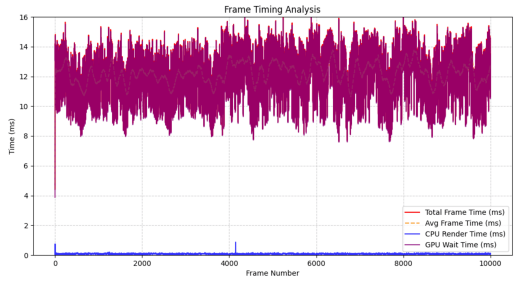

Figure 10: Frame Timing Analysis - OpenGL-SC

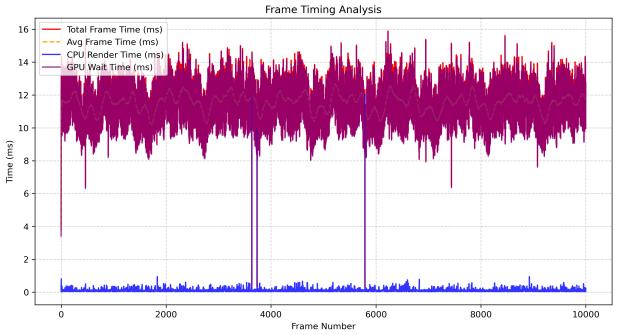

Figure 11: Frame Timing Analysis - OpenGL Desktop

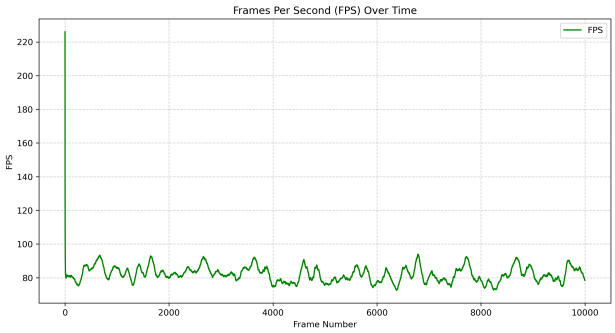

Figure 12: Frames Per Second - OpenGL-SC

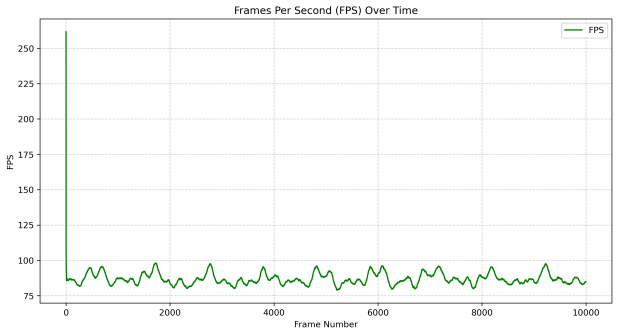

Figure 13: Frames Per Second - OpenGL Desktop

4.3. Lighting and Shadows

Vertex Shader for SC/ES: For our third experiment, we wanted to test the two versions of OpenGL against a more classic usage of Graphics Engine: a simple scene, with a model in the scene, lights spread around and shadows casted by the objects.
The reasoning behind this decision is to evaluate

- Vertex and Fragment shader's operations, as lighting and shadows calculation are completely offloaded to the shaders.
- Accuracy and fidelity from the rendering point of view (artefacts in the rendered scene, precision of the shadows' casted)
- Using Vertex Array Objects with different data layout vs. using multiple Vertex Buffer Objects and Element Buffer Objects, redefining the layout of the data for each draw call.

For what concerns the last point, the idea is to define the objects in our scenes with different layout for the data distributions (meaning we define an object with vertex + normals + color coordinates and another with only vertex + color). In a normal OpenGL Desktop appication, we can simply bind the different VBOs, with differnt layouts, to several VAOs and bind them at run time before drawing the specific object. In OpenGL-SC we instead have to redefine the layout of the data before drawing any object whose data layout differs from the previously rendered object.

```
1   // OpenGL Desktop
2
3   // Init phase
4   glGenVertexArrays(1, &VAO);
5   glGenBuffers(1, &VBO);
6   glGenBuffers(1, &EBO);
7
8   glBindVertexArray(VAO);
9
10  glBindBuffer(GL_ARRAY_BUFFER, VBO);
11  glBufferData(GL_ARRAY_BUFFER, sizeof(wall_vertices), wall_vertices, GL_STATIC_DRAW)
    ;
12
```

```
    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 9 * sizeof(float), (void*)0);
    glEnableVertexAttribArray(0);
    glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 9 * sizeof(float), (void*)(3 * sizeof(float)));
    glEnableVertexAttribArray(1);
    glVertexAttribPointer(2, 3, GL_FLOAT, GL_FALSE, 9 * sizeof(float), (void*)(6 * sizeof(float)));
    glEnableVertexAttribArray(2);

    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO);
    glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(wall_indices), wall_indices, GL_STATIC_DRAW);

    [...]

    // Rendering loop
    glBindVertexArray(VAO);
    s.setMatrix("model", model_matrix);
    glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_INT, 0);



    // OpenGL- SC

    // Init phase
    glGenBuffers(1, &VBO);
    glGenBuffers(1, &EBO);

    glBindBuffer(GL_ARRAY_BUFFER, VBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(wall_vertices), wall_vertices, GL_STATIC_DRAW);

    posLoc = glGetAttribLocation(shader.ID, "aPos"); // this substitute layout numbering system for glsl 330
    normLoc = glGetAttribLocation(shader.ID, "aNormal");
    colorLoc = glGetAttribLocation(shader.ID, "aColor");

    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO);
    glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(wall_indices), wall_indices, GL_STATIC_DRAW);

    [...]

    // Rendering loop
    glBindBuffer(GL_ARRAY_BUFFER, VBO);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO);
    // vertex position
    glVertexAttribPointer(posLoc, 3, GL_FLOAT, GL_FALSE, 9*sizeof(float), (void *)0);
    glEnableVertexAttribArray(posLoc);
    // vertex color
    glVertexAttribPointer(colorLoc, 3, GL_FLOAT, GL_FALSE, 9 * sizeof(float), (void *)(6*sizeof(float)));
    glEnableVertexAttribArray(colorLoc);
    // vertex texture
    glVertexAttribPointer(normLoc, 3, GL_FLOAT, GL_FALSE, 9 * sizeof(float), (void *)(3 * sizeof(float)));
    glEnableVertexAttribArray(normLoc);
    glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_SHORT, 0);
}
```

It is obvious that the main difference is that for OpenGL-SC we need to explicitly set the layout of the data for the GPU for every frame, instead we can store it in VAOs for OpenGL Desktop. It is clear that programs in which data structures are layered in a constant format are better suited for OpenGL-SC, but sometimes it is inevitable to perform such restructuring between frames (even though it is suggested to minimize it as much as possible).

In our scene, we included **Ambient Light**, **Point Lights** and **Spot Light**. Point lights varied in number during out tests: we experimented with 6, 50 and 100 point lights in the scene. Shadows are calculated only from the perspective of the spotlight, in order for us to better get a visual understanding of the scene (making sure shadows are consistent within our model, easing our job, being this a qualitative observation rather than an objective one). During our tests, we moved around the scene to simulate a real-life application usage. We tried to keep the movement performed by the camera as similar as possible between iterations.
The image below represents the scene we tested, with 6 point lights (the colored cubes), a model in the center of the scene (the backpack) and lights/shadows inclusion.

Figure 14: Third Test Scene

One major difference between the two versions regards the quality of the shadows. In OpenGL-SC, the floating point number precision is static and set at the beginning for each shader. This is crucial for shadow quality. Since Safety Critical applications are usually deployed on a hardware with limited resources, floating point operations might be less precise with respect to the Desktop version. This can create an effect called **Shadow Acne**. We won't delve into this problem, since it mostly concerns topics outside of the scope of this report, but following are two images comparing the quality of the shadows in the two programs.

Figure 15: Model and Shadow - OpenGL-SC

Figure 16: Model and Shadow - OpenGL Desktop

Looking at the pictures, dark lines on the backpack are visible in the SC version (also in the standard program, but less evidently). Those are the result of Shadow acne and floating point precision (for the bias term in shadow calculation) and limitation in SC applications.
Other than these small qualitative differences, the two applications also diverge when handling multiple light sources. The shader used for the Desktop version for handling lights is pretty standard, so below we include only an example of shader used for the SC version.

```
1 #version 100
2 // Fragment shaders in GLES require a default precision
```

```
precision mediump float;

#define MAX_LIGHTS 100

struct DirectionalLight{
    vec3 direction;
    vec4 ambient;
    vec4 diffuse;
    vec4 specular;
};
uniform DirectionalLight directionalLight;

struct Light{
    vec3 position;
    vec4 color;
};
uniform Light pointLights[MAX_LIGHTS];

struct SpotLight{
    vec3 position;
    vec3 direction;
    float cutOff;
    float outerCutOff;
    vec4 color;
};
uniform SpotLight spotLights[2];

uniform vec3 viewPos;
uniform sampler2D shadowMap;

varying vec3 Normal;
varying vec3 FragPos;
varying vec3 Color;
varying vec4 FragPosLightSpace;


float ShadowCalculation(vec4 fragPosLightSpace){
    vec3 projCoords = fragPosLightSpace.xyz / fragPosLightSpace.w;
    projCoords = projCoords * 0.5 + 0.5;

    if(projCoords.z > 1.0)
        return 0.0;

    float closestDepth = texture2D(shadowMap, projCoords.xy).r;
    float currentDepth = projCoords.z;

    float bias = 0.001;
    float shadow = currentDepth - bias > closestDepth ? 1.0 : 0.0;

    return shadow;
}

void main(){
    vec3 ambient = vec3(0.0);
    vec3 diffuse = vec3(0.0);
    vec3 specular = vec3(0.0);

    vec3 norm = normalize(Normal);
    vec3 viewDir = normalize(viewPos - FragPos);

    // Ambient Light
    ambient += directionalLight.ambient.w * directionalLight.ambient.xyz * Color;

    // Point Lights
    for (int i = 0; i < MAX_LIGHTS; i++) {
        vec3 lightDir = normalize(pointLights[i].position - FragPos);
```

```
69          float distance = length(pointLights[i].position - FragPos);
70          float attenuation = 1.0 / (1.0 + 0.07 * distance + 0.017 * (distance * distance
    ));
71
72          // Diffuse
73          float diff = max(dot(norm, lightDir), 0.0);
74          diffuse += (diff * pointLights[i].color.xyz * Color) * attenuation;
75
76          // Specular
77          vec3 reflectDir = reflect(-lightDir, norm);
78          float spec = pow(max(dot(viewDir, reflectDir), 0.0), 32.0);
79          specular += (spec * pointLights[i].color.w * pointLights[i].color.xyz) *
    attenuation;
80      }
81
82      // Spot Light
83      vec3 spotLightDir = normalize(spotLights[0].position - FragPos);
84      float spotDistance = length(spotLights[0].position - FragPos);
85      float spotAttenuation = 1.0 / (1.0 + 0.022 * spotDistance + 0.0019 * (spotDistance
    * spotDistance));
86
87      float theta = dot(spotLightDir, normalize(-spotLights[0].direction));
88      float epsilon = spotLights[0].cutOff - spotLights[0].outerCutOff;
89      float intensity = clamp((theta - spotLights[0].outerCutOff) / epsilon, 0.0, 1.0);
90
91      // Diffuse
92      float spotDiff = max(dot(norm, spotLightDir), 0.0);
93      diffuse += (spotDiff * spotLights[0].color.xyz * Color) * spotAttenuation *
    intensity;
94
95      // Specular
96      vec3 spotReflectDir = reflect(-spotLightDir, norm);
97      float spotSpec = pow(max(dot(viewDir, spotReflectDir), 0.0), 32.0);
98      specular += (spotLights[0].color.w * spotSpec * spotLights[0].color.xyz) *
    spotAttenuation * intensity;
99
100     // Final Calculation
101     float shadow = ShadowCalculation(FragPosLightSpace);
102     vec3 result = ambient + (1.0 - shadow) * (diffuse + specular);
103
104     gl_FragColor = vec4(result, 1.0);
105 }
```

One main aspect that makes this shader differ from a standard implementation is that loops must have a number of iterations defined at compile time. Therefore, we defined a MAX_LIGHT variable that indicates the maximum number of point lights that can be placed in the scene. In a standard application, we would also use an uniform integer to indicate the exact number of lights currently present, however, this is not acceptable in SC applications.
The need to redefine the layout of the data, shadow calculations (which are notoriously heavy for the GPU) and the multiple lights put a heavy toll on our SC program, which results in very unstable results. This is obviously justifiable by the fact that we are merely simulating SC behavior using OpenGL-ES instead, but at the same time SC is known to work poorly in this specific scenario, thus the results do not come as a surprise.

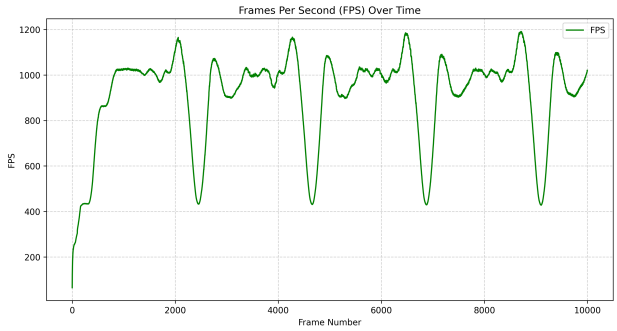

Figure 17: FPS vs Frame Number - OpenGL-SC

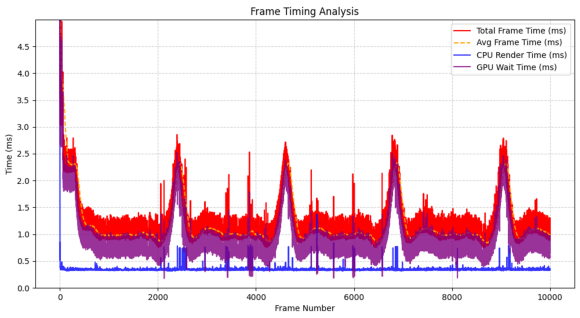

Figure 18: Frame Timing Analysis - OpenGL-SC

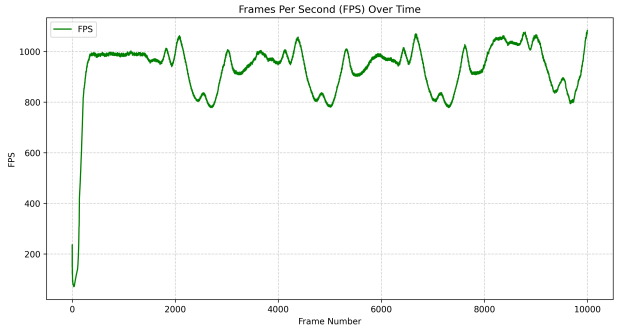

Figure 19: FPS vs Frame Number - OpenGL Desktop

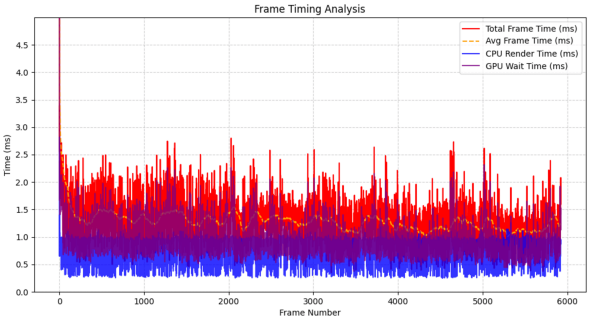

Figure 20: Frame Timing Analysis - OpenGL Desktop

It is evident how SC is not well suited for a situation like this one. It is no wonder that many SC applications are developed with low quality graphics in mind; shadows are usually not taken into consideration, as well as complex scenes with multiple lights. WCET in this context is also not meaningful at all as the execution time was voluntarily worsened for the purpose of the test.

5. Conclusion

In conclusion, this work has provided a comprehensive comparison between Desktop OpenGL and its Safety-Critical (SC) counterpart, analyzing their theoretical differences and validating them through targeted experiments. By simulating SC behavior with an OpenGL ES 2.0–compliant approach, we examined how constraints on features such as VAOs, instanced rendering, shader compilation, and modern GLSL functionalities impact both performance and predictability. Our experiments confirmed several of our initial expectations.

- **Performance vs. Stability:** We anticipated that OpenGL SC would deliver lower raw performance compared to Desktop OpenGL, and this was clearly observed: in the "World of Cubes" test, SC lagged behind its Desktop counterpart by more than 100 FPS at 10k instances. However, SC maintained a far more stable frame rate, while Desktop OpenGL showed noticeable variability. This stability aligns directly with the SC design goal of predictable timing.
- **Feature limitations:** We expected the absence of VAOs, instancing, and modern GLSL features to increase code verbosity and reduce convenience. This was also borne out in practice: the SC versions required more explicit buffer and attribute management and repeated draw calls. Although this added overhead, it made the rendering flow fully visible in code and logs, which is essential for certification.
- **Desktop unpredictability:** Another clear result was that the FPS gap between Desktop and SC implementations did not always scale linearly with workload size. Also WCET analysis is almost impossible to achieve accurately on desktop, while for SC implementations it is quite straight forward and can be directly measured without problems. For instance, in the 1M cube test, Desktop OpenGL achieved only 7 FPS against 5 FPS for SC. The difference here was smaller than expected, suggesting that instancing loses efficiency at very large scales due to hidden GPU bottlenecks, while SC's explicit loop remained consistent.

These findings highlight the trade-off that defines the two APIs: Desktop OpenGL offers higher throughput and flexibility, especially with advanced features like instancing and compute shaders. Instead SC deliberately

sacrifices peak efficiency in favor of determinism, stability, and certifiability. The trade-off is not accidental but intrinsic to their target domains: consumer graphics prioritize speed, while safety-critical systems demand transparency and analyzable timing.
From a design perspective, developers targeting SC environments must embrace a more static and predictable programming model, be prepared for potential performance penalties, and carefully adapt their code rather than relying on direct substitution of Desktop features. Our results reinforce the idea that in safety-critical contexts, "slower but stable" is not a drawback, but rather a necessary requirement for certifiable graphics pipelines.

5.1. Code and Repository

All the code used for the experiments and environment testing is available into a repository created by us and made public. We had decided to split the experiments from an implementation point of view as well as the workspace: each is found inside a different branch of the repository. Here are the links to all the different branches and what they hold:

- **Main repository / Experiment 1 in OpenGL Desktop**: World of Cubes Desktop
- **Experiment 1 in OpenGL SC**: World of Cubes SC
- **Experiment 2 in OpenGL Desktop**: Compute Shaders Desktop
- **Experiment 2 in OpenGL SC**: Compute Shaders SC
- **Experiment 3 in OpenGL Desktop**: Lightning and Shadows Desktop
- **Experiment 3 in OpenGL SC**: Lightning and Shadows SC

References

[1] CoreAVI. Opengl sc 1.0 and 2.0: White paper. https://coreavi.com/wp-content/uploads/2018/08/OpenGL2_White-Paper_2016.pdf.

[2] Joey de Vries. Learnopengl. https://learnopengl.com/.

[3] KDAB. Opengl sc: A profile for safety critical systems. https://www.kdab.com/documents/KDAB_whitepapers_OpenGLSC_march2017-1_7zpSAiF.pdf.

[4] Khronos Group. Opengl 3.3 core profile specification. https://registry.khronos.org/OpenGL/specs/gl/glspec33.core.pdf, .

[5] Khronos Group. Opengl sc – safety critical graphics. https://www.khronos.org/openglsc/, .

[6] Khronos Group. Opengl sc 2.0.1 specification. https://registry.khronos.org/OpenGL/specs/sc/sc_spec_2.0.1.pdf, .

[7] Khronos Group. Vertex specification — opengl wiki. https://www.khronos.org/opengl/wiki/Vertex_Specification, .

[8] YouTube. Compute shader / menger sponge (video). https://www.youtube.com/watch?v=TJoa_xUoV4w, .

[9] YouTube. Opengl sc overview (video). https://www.youtube.com/watch?v=zhJdozJcaOs, .